React 18 · Vite 6 · Tailwind 4 · Mock API

JobKhoj

Technical Documentation & Technology Report

Full-stack job portal — job seekers, recruiters and administrators in one platform.

Document type	Frontend	Backend layer	Date
Project / Technology Report	**React + Vite (SPA)**	**Mock data + REST ready**	**06 October 2026**

✓ 241 automated checks passing · ✓ Lint clean · ✓ Production build successful

1. Project Overview

JobKhoj is a single-page application implementing a complete job-portal workflow for three roles: job seeker, recruiter and administrator. The interface, business rules, routing and data layer are fully built; the persistence layer currently runs on a seeded in-memory mock database that mirrors a REST API, so the app can be switched to a live Spring Boot backend by changing one environment variable.

ROUTES	PAGE COMPONENTS	REUSABLE COMPONENTS
43	**38**	**48**
Across 4 access shells	+ 6 layouts	Kit + charts + domain

SERVICE MODULES	MOCK LAYER FILES	AUTOMATED CHECKS
10	**17**	**241**
Single API boundary	DB, handlers, seed	Data + render probes

1.1 Module completion status

Module	Pages	Status
Public / marketing & discovery	7	**Complete** — Home, job search, job details, company directory, company details, 404, 403
Authentication	4	**Complete** — login, register, forgot password, reset password
Job seeker	9	**Complete** — dashboard, profile, resume, applications, saved jobs, alerts, notifications, settings
Recruiter	12	**Complete** — dashboard, company profile, job CRUD, pipeline, candidates, analytics, settings
Administrator	7	**Complete** — dashboard, users, recruiters, companies, jobs, applications, reports

2. Technology Stack

2.1 Runtime & core libraries

Technology	Version	Used for	Notes
React	18.3.1	UI library	Function components, hooks, `lazy()` code-splitting, `Suspense`
React DOM	18.3.1	DOM renderer	Also used by the automated render harness (`react-dom/server`)
React Router DOM	6.28.1	Client-side routing	Nested layout routes, `protected` / `role` guards, URL-param filters
Axios	1.7.9	HTTP client	Interceptors for auth token, 401 handling and base URL; drives *live* mode
React Hook Form	7.54.2	Form state	Login, register, profile, job form, company profile, settings
Recharts	3.10.1	Data visualisation	Line, bar, donut and horizontal-bar charts for recruiter & admin analytics
Lucide React	0.469.0	Icon system	Tree-shaken icon set used across the UI

2.2 Build, styling & quality tooling

Technology	Version	Used for	Notes
Vite	6.0.7	Dev server & bundler	Port 5173, hot module replacement, manual chunk splitting for React and charts
@vitejs/plugin-react	4.3.4	React transform	JSX / Fast Refresh support
Tailwind CSS	4.0.0	Utility-first styling	Integrated through `@tailwindcss/vite` (no separate PostCSS config)
ESLint	9.17.0	Static analysis	Flat config (`eslint.config.js`) with React, Hooks and Refresh rule sets
esbuild `(transitive)`	—	Test bundling	Compiles the smoke suite, alias `@ → src` , automatic JSX

2.3 Path alias & environment

Item	Value

Source alias	`@ → ./src` (Vite `resolve.alias`, mirrored by esbuild `--alias:@=./src`)
API mode	`VITE_API_MODE` = `mock` (local database) or `live` (REST)
API base URL	`VITE_API_BASE_URL` = `http://localhost:8080/api` (Spring Boot)
Storage namespace	Every key is prefixed `jobkhoj:` so the app never collides with other data on the origin

> **Backend note:** the frontend is written against a service layer, not against mock code directly. Switching `VITE_API_MODE=live` sends every call through Axios to the Spring Boot API without touching a single page component.

3. Architecture

The code follows a layered structure: pages never touch storage directly — they call services, the service layer chooses mock or live, and mock handlers own all business rules.

```
- Page / Component  (React, hooks, rendering)
        ↓  module import
- Service layer    (authService, jobService, adminService …)
        ↓  picks mode at runtime
   → mock handler  → in-memory DB  ↓  (seed data)
   → Axios client → http://localhost:8080/api
```

3.1 Folder structure

```
src/
├── components/    48 files   common kit, charts, jobs, seeker, recruiter, layout
├── pages/         40 files   admin(7) recruiter(12) seeker(9) auth(4) jobs(2)
│                             companies(2) errors(2) Home
├── services/      10 files   API boundary + delegation to mock or live
├── mock/          17 files   db.js, handlers/ (11), seed/ (6)
├── layouts/        6 files   Public, Seeker, Recruiter, Admin, Dashboard, Auth
├── hooks/          9 files   useFetch, useAuth, useToast, usePagination …
├── routes/         4 files   AppRoutes, paths, ProtectedRoute, RoleRoute
├── context/        4 files   AuthProvider, ToastProvider + contexts
└── utils/          4 files   constants, format, storage, cn
```

3.2 Layers in detail

Layer	Responsibility	Representative files
Pages	Route-level screens; fetch, render, handle user intent	`pages/admin/Reports.jsx`, `pages/jobs/JobSearch.jsx`
Components	Reusable UI: tables, modals, forms, charts, skeletons	`DataTable`, `ConfirmDialog`, `Charts.jsx`
Services	One import surface per domain; mock/live switch lives here	`jobService.js`, `adminService.js`, `api.js`
Mock handlers	Business rules: filtering, pagination, moderation, pipeline moves	`handlers/jobs.js`, `handlers/admin.js`
Seed	Deterministic demo dataset — companies, jobs, users, analytics	`seed/index.js` + 5 domain seeds
Constants	Single source of truth for roles, statuses, tones, route paths	`utils/constants.js`, `routes/paths.js`

3.3 Access control

Guard	Behaviour
`<ProtectedRoute>`	Any signed-in user; otherwise redirects to login
`<RoleRoute roles={[…]}>`	One specific role; otherwise redirects to that role's 403 / home
Home routing by role	`JOB_SEEKER → /dashboard`, `RECRUITER → /recruiter/dashboard`, `ADMIN → /admin`

4. Feature Modules

4.1 Public & discovery

Route	Page	Features
`/`	Home	Hero search, featured jobs, skill shortcuts, top verified companies, employer CTA
`/jobs`	Job search	Full-text search, faceted filters, URL-synced params, sorting, pagination
`/jobs/:id`	Job details	Description, skills, similar jobs, save/apply actions, share, view tracking
`/companies`	Company directory	Verified-first listing, search & verification filters, paginated
`/companies/:slug`	Company profile	About, benefits, key facts, open roles
`/login` `/register` `/forgot-password` `/reset-password`	Auth (4)	Role-aware sign-in and registration (seeker / recruiter), validation
`/403` `*`	Errors (2)	Forbidden and not-found screens

4.2 Job seeker

Route	Page	Features
`/dashboard`	Dashboard	Application stats, recommended jobs, recent activity
`/dashboard/profile`	Profile	Personal details, skills, experience, education, profile completeness
`/dashboard/resume`	Resume	Resume upload / management
`/dashboard/applications`	My applications	Application list with status badges and filters
`/dashboard/applications/:id`	Application details	Status timeline, job summary, withdrawal
`/dashboard/saved-jobs`	Saved jobs	Saved / shortlisted job list
`/dashboard/job-alerts`	Job alerts	Create and manage keyword alerts
`/dashboard/notifications`	Notifications	Typed inbox with icons and read state
`/dashboard/settings`	Settings	Password change, notification preferences

4.3 Recruiter

Route	Page	Features
`/recruiter/dashboard`	Dashboard	Company metrics, open roles, recent applications
`/recruiter/company`	Company profile	Edit company record, verification badge
`/recruiter/jobs` `/jobs/create` `/jobs/:id/edit` `/jobs/:id`	Job CRUD (4)	Create, edit, publish, close; shared `JobForm`
`/recruiter/jobs/:id/applicants`	Applicants	Kanban pipeline, drag-and-drop stage moves, list view
`/recruiter/applicants/:applicationId`	Applicant details	CV summary, stage actions, rejection/offer confirmation
`/recruiter/candidates`	Candidates	Cross-job candidate search and filters
`/recruiter/analytics`	Analytics	Funnel, source and status charts
`/recruiter/settings`	Settings	Recruiter account and notification preferences

4.4 Administrator

Route	Page	Features
`/admin`	Dashboard	Platform KPIs, moderation queues, shortcuts, charts
`/admin/users`	Users	Search, role/status filters, block, unblock, delete
`/admin/recruiters`	Recruiters	Role and status filtering, linked company names, moderation
`/admin/companies`	Companies	Verify / reject / suspend with confirmation
`/admin/jobs`	Jobs	Approve, reject, close; application counts
`/admin/applications`	Applications	Read-only cross-platform application monitoring
`/admin/reports`	Reports	Status splits, trends, CSV export, data reset

5. Data & Mock Layer

A deterministic in-memory database behaves like a REST API, so the UI can be developed and tested before the backend exists.

Area	Files	What it does
Database	`mock/db.js`	In-memory tables with find / where / update / insert primitives
Seed data	`seed/index.js` + `companies`, `jobs`, `users`, `applications`, `analytics`	Deterministic demo dataset: employers, verified/pending status, jobs, seeker, recruiter, admin, applications, metrics
Handlers	`handlers/` — auth, jobs, companies, applications, users, recruiter, admin, query, index, latency	Business rules: search & facets, pagination, moderation transitions, pipeline stage moves, role checks, artificial latency

5.1 Domain rules implemented in the mock layer

- Moderation lifecycle — companies *Pending → Verified / Rejected / Suspended*, jobs *Pending Approval → Active / Rejected*, users *Active → Blocked*.
- Recruiter Kanban pipeline — *Applied → Screening → Shortlisted → Interview → Selected → Hired*, with a terminal *Rejected* column counted separately.
- Status change updates both the recruiter's stage and the seeker's visible status, and appends a timeline entry.
- Blocked accounts are excluded from dashboards and queues.
- Company ownership is resolved from the session, not from the request body, so a recruiter cannot edit another company.

5.2 Status vocabulary (single source of truth)

Constant group	Values
`ROLES`	`JOB_SEEKER`, `RECRUITER`, `ADMIN`
`APPLICATION_STATUS`	Applied, Under Review, Shortlisted, Interview, Rejected, Hired
`JOB_STATUS`	Draft, Pending Approval, Active, Closed, Rejected, Expired
`COMPANY_STATUS`	Pending, Verified, Rejected, Suspended
`USER_STATUS`	Active, Blocked
`PIPELINE_STAGE`	APPLIED, SCREENING, SHORTLISTED, INTERVIEW, SELECTED, HIRED, REJECTED

`NOTIFICATION_TYPE`	SHORTLISTED, INTERVIEW, MATCH, STATUS

> **Enforced by tooling:** an ESLint rule bans these values when written as raw string literals, so status values can no longer drift away from the constants. This closed a real class of bug where `'VERIFIED'` (uppercase) was written while the constant is `'Verified'` — which silently returned zero rows from every filter.

6. Quality Assurance & Scripts

6.1 npm scripts

Command	Purpose	Detail
`npm run dev`	Development server	Vite on port 5173, opens automatically
`npm run build`	Production build	Outputs `dist/` with split vendor chunks
`npm run preview`	Preview the build	Serves the production bundle locally
`npm run lint`	Static analysis	ESLint flat config across all JS/JSX
`npm run smoke`	Automated test suite	241 assertions — data contracts + component render probes

6.2 What the smoke suite covers

Suite	Examples of what is asserted
Search & filtering	Keyword match, location/skill filters, facets, sorting, pagination contracts
Authentication	Session login, token decode, role-based home, unauthorised access
Application pipeline	Stage moves, status synchronisation, timeline entries, rejection recovery
Admin moderation	Company verify/reject, job approval, user block/unblock, queue counts
Reports & charts	Status splits sum to 100%, chart series shapes, CSV export
Service layer	Mock/live delegation, session ownership, not-found errors
Render probes	**All 44 route components mounted** — catches invalid element types, hook-order errors and unguarded render expressions

6.3 Verification results

Check	Result	Detail
Automated smoke suite	**241 / 241**	0 failures
ESLint	**Clean**	0 errors, 0 warnings
Production build	**Passing**	Vite build completes without errors

Route inventory	**43 / 43**	Every route resolves to an existing component with a default export
Placeholder scan	**None**	No stubbed or unfinished screens remain

6.4 Known limitation

> There is no browser-level (end-to-end) test harness in this project. The automated suite verifies data behaviour and that every screen renders, but interactive flows such as clicking through an application, submitting forms or drag-and-drop have not been exercised by an automated browser test.

7. Delivery Summary

Area	Status
UI framework & tooling	React 18 + Vite 6 + Tailwind CSS 4 + ESLint 9 (flat config)
Routing & access control	React Router 6 with nested layouts, protected routes and role guards
Forms & validation	React Hook Form across auth, profile, job and company forms
Charts	Recharts — line, bar, donut and horizontal bars
Backend integration	Service layer with mock / live switch; Axios ready for Spring Boot
Feature completeness	All 5 modules (public, auth, seeker, recruiter, admin) complete
Test coverage	241 automated checks: data contracts + all 44 component renders
Code quality	Lint clean; status literals enforced against shared constants
Build	Production build successful and code-split

How to run

```
npm install        # install dependencies
npm run dev        # start development server (http://localhost:5173)
npm run lint       # static analysis
npm run smoke      # run all 241 automated checks
npm run build      # production build to dist/
```

Demo accounts

Role	Email	Password
Job seeker	`seeker@jobkhoj.in`	Any value (mock accepts any)
Recruiter	`recruiter@jobkhoj.in`	Any value (mock accepts any)
Administrator	`admin@jobkhoj.in`	Any value (mock accepts any)